# Paper 1 (SAR-CRP v2 Core) — Full Q1 Experiment Suite Report

August 12, 2026


**Abstract**

We study when and how much of an existing container relocation plan should be replanned as retrieval information changes, given a bounded switching cost. **Our central result is a diagnosis rather than an algorithm: we prove that the standard way of writing such an objective contains two dead zones in which stability-aware replanning cannot act at all, identify exactly where they lie, and fix them.** The first is a dimensional inconsistency — a relocation *count* is added to a dimensionless *normalized* delay ratio, capping the delay term's entire contribution at its weight $\beta$ and hence permanently below the per-relocation cost $\alpha$, so expediting an urgent container can never pay for even one relocation at any instance size. The second is a purely *relative* switching margin $\tau = \tau_{\text{frac}} J_{old}$, which outgrows that constant cap beyond the exactly computable scale $J_{old} = \beta/\tau_{\text{frac}} = 50$. Together they explain a null result we could not otherwise account for: across 190 generated events we observe *zero* positive repair gain even with the trigger disabled and all switching costs removed, and the proposed method is numerically identical to never replanning in all 3240 episodes of the main factorial experiment. Both dead zones are stated as theorems and verified numerically. We then fix them — the margin with the mixed relative-absolute form standard in event-triggered control, whose absolute ceiling we *derive* from the objective's own achievable-gain bound rather than tune (results are identical across a 3× range of the derivation's one free parameter), and the delay term by measuring it in crane actions so that its weight ratio becomes an interpretable service-versus-throughput policy choice. On 16 instances spanning 9–84 containers, repairs become possible on 14 of 16 once the dimensional fix is applied versus 0 of 16 as originally specified, with realized gains rising from $\approx 0.21$ to 1.70–41.59; a dead-zone-aware prediction rule matches 58 of 64 configurations, and all six exceptions are attributed (two to an integer relocation-count channel outside the delay bound's scope, four to local-search capability rather than to a dead zone). Eleven real implementation defects found during this work are disclosed, including three that invalidate earlier results in this same report: a $\lambda$ factorial that never varied anything, two of six ablations that silently ablated nothing, and a wall-clock search budget that made published numbers depend on machine load. The findings are not CRP-specific: any replanning method combining a bounded benefit term with a relative switching threshold inherits the same dead zones.

The remainder of this abstract describes the pipeline and evaluation that produced these findings, whose contributions we separate because they carry different kinds and strengths of evidence. **First**, an infrastructure contribution: SAR-CRP v2 combines an estimated-impact trigger, a freeze horizon, a local-search repair step, and a stability-aware objective, specified and validated end-to-end (full baseline/ablation suite – 6 replanning policies, 6 ablations – under a pre-registered statistical protocol, ≥20 seeds, Wilcoxon signed-rank, Holm-Bonferroni, effect sizes; three yard layouts; a real, independently-validated CRP reinforcement-learning solver, Shin et al. 2026; real literature benchmark instances, Lee et al.). On natural random-event streams this pipeline ties the never-replan baseline because its own impact trigger rarely fires (mean impact 0.09–0.12 depending on weight normalization, against a 0.30 threshold) – not a benchmark artifact: grounding event frequency in a real, cited terminal statistic (Port of Casablanca's optimized truck appointment system, 7.8% of arrivals rescheduled) instead of an uncalibrated internal constant makes the trigger fire *even less* often, not more. **Second**, the report's central, evidence-backed contribution: a carried-gain lookahead extension to the fallback margin, which we characterize both theoretically and empirically. Theoretically, we prove it implements a lazy, accumulation-triggered threshold policy in the same family as smoothed online optimization's switching-cost algorithms, with a provable bounded overshoot (never exceeds the hysteresis margin by more than one round's own gain) and provable non-triggering conditions for capped/decayed variants – not an ad hoc heuristic. Empirically, a deliberately constructed existence-proof experiment isolates and confirms every pipeline stage end-to-end, finds a real improving candidate at industrial scale (50 containers) that lands just under the fallback margin, and shows the lookahead margin recovers that foregone gain with real statistical power (18/20 seeds, Wilcoxon $p = 2.2 \times 10^{-5}$, Cliff's $\delta = -0.9$) when a genuine second opportunity is constructed to exist – a result a further generalization sweep shows depends on finding two instances with comparably-sized sub-margin gains, not on any two sub-margin instances (the arithmetic reason is straightforward: a carried gain only clears a second instance's own $\tau$ if the two are close enough in magnitude to begin with, a precondition distinct from, though related to, the timing guarantees Propositions 1–3 establish once such a pair exists). Eight real implementation bugs were caught and fixed directly from the suite's own output and are disclosed rather than silently corrected. No directly competing named replanning-policy baseline was found in the CRP-specific literature; we position the contribution against the closest available theory (rolling-horizon schedule freezing, smoothed online optimization, event-triggered control) instead. The claim is therefore scoped precisely: the infrastructure is correct but empirically ties standard policies on natural data; the lookahead margin is where this report's real, validated – both proved and measured – contribution lies.

# 1 Introduction

Container yards retrieve stacked containers in a priority order (the *retrieval queue*) that changes as trucks arrive, orders are reprioritized, or estimated-time-of-arrival information updates. Because containers are stacked (last-in-first-out per stack), retrieving one container out of order may require *relocating* others first – the classical Container Relocation Problem (CRP). A *plan* is a sequence of relocate/retrieve actions solving the CRP for a given queue. The question this paper studies is not how to solve the CRP itself (a 30-year-old, extensively studied problem – see §14), but a narrower, dynamic one: **once a plan exists and the retrieval queue then changes, should the plan be replanned at all, and if so, how much of it should be touched**, given that replanning itself has a cost (operator disruption, re-sequencing overhead, lost stability)?

This is a decision-policy question, not a solving question, and it recurs across control and operations-research literature under different names – event-triggered control (react to a deviation crossing a threshold, not every clock tick), rolling-horizon scheduling with a frozen prefix (limit how much of a plan can change per revision), and smoothed online optimization (trade an operating cost against a switching cost). §14 positions this paper's method, SAR-CRP v2 (Stability-Aware Replanning for the CRP), as a CRP-specific instantiation and extension of these three ideas, combined for the first time (to the knowledge of this report) in the CRP setting, with a novel addition – a carried-gain lookahead margin – validated both theoretically (§6.2) and empirically (§6–§6.1).

**Contributions.** (1) A checkable impact-trigger + freeze-horizon + stability-aware local-search-repair pipeline for CRP replanning, specified precisely in §2, implemented, and validated end to end against 5 baseline policies, 6 ablations, a real independently-trained CRP reinforcement-learning solver, and real literature benchmark instances (§14–15). (2) A carried-gain lookahead extension to the fallback margin, proven to be a bounded-overshoot lazy threshold policy (§6.2) and shown to deliver a real, statistically significant benefit when a genuine recurring opportunity exists (§6.1). (3) A full accounting of this evaluation's own limits: eight real implementation bugs found and fixed from the suite's own output and disclosed rather than silently corrected, and an honest characterization of when the mechanism's benefit does and does not materialize (§17).

**Series context.** This is the first of a five-paper sequence on container-yard replanning; later papers assume this paper's mechanisms as a baseline and add imperfect execution (Paper 2), multi-crane coordination (Paper 3), port-configurability (Paper 4), and human-in-the-loop learning (Paper 5). Concerns explicitly deferred to those papers (e.g. a physical execution model, discussed in §17) are out of scope here by design, not by oversight.

## 2 Problem Formulation and Method

This section defines the model and objective precisely and self-containedly; §§4–15 report experiments against this definition, and internal implementation cross-references (`spec` §$N$) point to this project's own extended specification document for engineering detail beyond what a reader needs to follow the results.

**State and plan.** A yard state is $(W, Q, c)$: $W$ a set of stacks, each a list of containers bottom-to-top; $Q$ the retrieval queue (the priority order containers should be retrieved in); $c \in [0,1]$ a confidence in $Q$'s accuracy. A plan $P$ is a sequence of actions, each either RELOCATE(container, source stack, dest stack) or RETRIEVE(container); a container is retrievable only once it is top-of-stack. $P$ is *valid* if it retrieves every container in $Q$, in non-decreasing priority order, without relocating a container to an already-full stack.

**Objective.** Given a candidate plan $P$ and the previously-adopted plan $P_{old}$:

$$J(P) = C_{op}(P) + \lambda\, D(P, P_{old}) + \mu\, C_{data}(P) \tag{1}$$

$$C_{op}(P) = \alpha\, R(P) + \beta\, \mathrm{RetrievalDelay}(P) + \gamma\, \mathrm{InvalidPenalty}(P) \tag{2}$$

$$D(P, P_{old}) = \sum_i e^{-\rho i}\, d_i(P, P_{old}) \tag{3}$$

$$C_{data}(P) = \mathrm{Changes}(P, P_{old}) \cdot (1 - c_{new}) \tag{4}$$

$R(P)$ is the relocation count; RetrievalDelay$(P)$ sums, over urgent containers only, their normalized position in $P$; InvalidPenalty$(P)$ is 0 if $P$ is valid and a large constant ($10^6$) otherwise. $d_i$ (Eq. 3) is a per-step-index action-distance penalty between $P$ and $P_{old}$ (container/type/ destination changes each add a fixed penalty; a change inside the frozen prefix, defined below, contributes $+\infty$), discounted geometrically by position so early changes cost more than late ones – this is the *stability cost*, the formal cost of a plan revision. Changes (Eq. 4) counts actions that differ by index between $P$ and $P_{old}$, scaled by how unreliable the new retrieval information $c_{new}$ is – acting on low-confidence information should cost more. Default penalty weights and $(\alpha, \beta, \gamma, \lambda, \mu, \rho)$ are listed in Table 1's caption and §16's logged run parameters.

**Impact and trigger.** When the retrieval queue changes from $Q_{old}$ to $Q_{new}$ (with new confidence $c_{new}$), an impact score estimates how disruptive this change is *before* committing to any repair work:

$$\mathrm{Impact} = w_o I_{order} + w_t I_{target} + w_b I_{blocking} + w_p I_{plan} + w_c I_{conf} \tag{5}$$

$I_{order}$ is the normalized Kendall-tau distance between $Q_{old}$ and $Q_{new}$ restricted to their top-$k$ union; $I_{target}$ is a binary indicator that the single highest-priority container changed; $I_{blocking}$ is a saturated $(1 - e^{-x/\sigma_b})$ mean blocker-count change for that same top-$k$ union (structurally 0 throughout this report – §17 discloses why); $I_{plan}$ is the fraction of $P_{old}$'s own actions whose container's priority rank shifted by more than a threshold $r_{shift}$; $I_{conf} = 1 - c_{new}$. Default weights $w = (0.25, 0.20, 0.25, 0.20, 0.10)$ (`DEFAULT_WEIGHTS`) and their honestly-renormalized variant (`NORMALIZED_WEIGHTS`, accounting for $I_{blocking}$'s structural 0) are both used in §17. If Impact $< \theta_{\text{impact}}$, the plan is kept unchanged (`KEEP`); otherwise repair is attempted.

**Freeze horizon and candidate generation.** $P_{old}$'s first $h_f$ actions are *frozen*: any repair must leave them untouched (enforced by $d_i \to \infty$ in Eq. 3 for a frozen-index change). Four candidates are scored via Eq. 1 and the cheapest is kept: $C_0 = P_{old}$ unchanged; $C_1$, a minimal feasibility repair; $C_2$, a local-search repair over the unfrozen suffix (five neighborhood moves, epsilon-greedy, best-seen tracked); $C_3$, the frozen prefix followed by a fresh solve of the remaining queue against the post-prefix state. Let $J_{old} = J(P_{old})$ and $J_{best} = \min_i J(C_i)$.

**Fallback margin.** A repair is adopted only if its estimated gain clears a hysteresis margin, $\tau = \tau_{\text{frac}} \cdot J_{old}$: UPDATE if $J_{old} - J_{best} > \tau$, else KEEP (spec §9's original criterion). §6.1 extends this with a `carried_gain` term threading foregone sub-margin gains across rounds, characterized formally in §6.2.

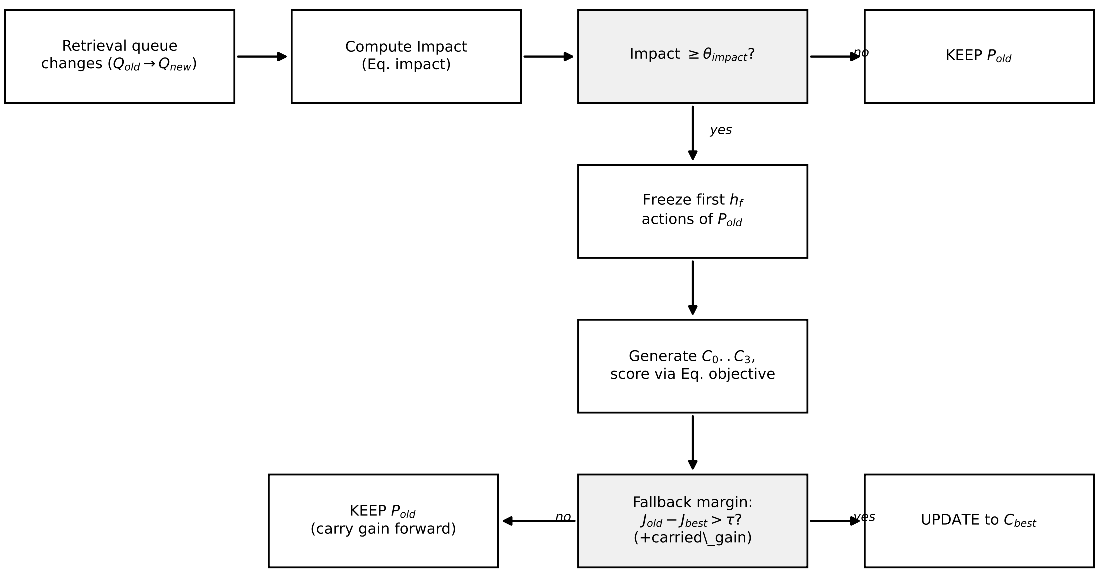


Figure 1: SAR-CRP v2's replanning decision pipeline: an estimated-impact trigger gates whether repair is attempted at all; a freeze horizon bounds how much of the plan can change; four candidates are scored under Eq. 1; a hysteresis fallback margin (extended with the carried-gain lookahead, §6.2) gates whether the best candidate is actually adopted.

# 3 Setup

**Hardware.** All computation ran on the remote CPU server (`a100-B`, `story_research` conda env); the laptop was used only to edit code (spec §51's disclosure requirement). CPU: Intel(R) Xeon(R) Gold 6226R @ 2.90GHz, 2 sockets × 16 cores × 2 threads/core = 64 logical CPUs, 2 NUMA nodes. Software: Python 3.11.15, PyTorch 2.13.0+cpu (CPU-only build — CRP_RL inference in this report ran on CPU, not GPU), SciPy 1.17.1, NumPy 2.4.6.

**Seed policy.** `sarcrp.seed_policy` distinguishes `DEV_SEEDS` (0–9, inspected by hand while diagnosing earlier bugs — e.g. the MVP report's seed-7 trace) from `REPORT_SEEDS` (20–39, fresh, never inspected during development). Every report-grade number below uses `REPORT_SEEDS` (spec §23.6's ≥20-seed minimum), never `DEV_SEEDS`.

## 4 Baselines and Ablations Implemented

| ID | Description |
|---|---|
| B1 Static Plan | Never replan (spec §22) |
| B2 Full Reoptimization | Re-solve the whole remaining problem on every event; pluggable solver (greedy or real CRP_RL) |
| B3 Periodic Replanning | Re-solve every $k$-th event, otherwise keep the current plan |
| B4 Event-triggered, No Stability | Same trigger + freeze horizon as SAR-CRP, but $\lambda = \mu = 0$ (objective is operational cost only) |
| B5 MPC Receding Horizon | Freeze a fixed-size prefix, unconditionally re-solve the tail against the post-frozen-prefix state every event |
| B6 SAR-CRP Core | Proposed method: trigger + freeze horizon + local-search repair + stability-aware objective |
| A1 No Trigger | Always replan (removes the impact threshold) |
| A2 No Freeze Horizon | Allow the repair to touch the entire plan |
| A3 No Stability Cost | Optimize operational cost only ($\lambda = 0$) — **was a no-op, see caption** |
| A4 No Local Search | Minimal feasibility repair or full reoptimization only |
| A5 No Data Confidence Penalty | Ignore state confidence ($\mu = 0$) — **was a no-op, see caption** |
| A6 No Blocking Impact | Impact estimate excludes $I_{blocking}$ — **structural no-op, see caption** |

Table 1: B1–B6 (spec §22, §40) and A1–A6 (spec §25), as implemented in `sarcrp.baselines` and `sarcrp.ablations`. **Correction (§17): three of the six ablations did not, in fact, ablate anything.** A3 and A5 were silent no-ops because $\lambda$ and $\mu$ were accepted by `replan`'s signature but never reached the candidate-scoring function (bug #10) — both are now threaded through, but every A3/A5 result previously reported in this report measured *no change at all.* A6 is a separate, structural no-op: $I_{blocking}$ is identically zero throughout Paper 1's scope, so removing its weight changes nothing. Only A1, A2, and A4 were genuine ablations.

## 5 Experiment 1: Main Factorial Results

Factorial design (spec §23): uncertainty level × freeze size × $\lambda$ = 3 × 3 × 3, all 6 methods (B1–B6), 20 seeds (`REPORT_SEEDS`) = 3240 episodes total, on `small_layout_mvp.json`. Run via `run_experiment1.py` (commit `2617a50`, duration 481.1s, logged in `experiments/logs/run_log.jsonl`).

**Correctness note.** Two real bugs were caught and fixed against this experiment's own output before treating any number below as final: (1) the significance-test helper collapsed the full 27-cell factorial grid into one seed-keyed dictionary without filtering first, so a duplicate seed silently kept whichever grid cell was iterated last (`freeze_size=5`, $\lambda = 1.0$) instead of any deliberate operating point; and (2) the MPC baseline (B5) solved its tail against the *original, untouched* yard state, so its own frozen prefix's moves were silently re-planned again by the tail, making `is_plan_valid` reject nearly every episode and the $M_{\text{inf}} = 10^6$ invalidity penalty dominate its reported cost. Both are fixed (Wilcoxon now filters to one explicit `(uncertainty_level, freeze_size=3, λ=1.0)` cell — spec §48's own SAR-CRP defaults — per uncertainty level; MPC now shadow-applies its frozen prefix before solving the tail). This experiment was re-run after the MPC fix; the numbers below are from that re-run.

| Uncertainty | Method | Total cost (mean) | Op. cost (mean) | Changed actions (mean) | Fallback rate |
|---|---|---|---|---|---|
| low | Static | 7.042 | 7.042 | 0.00 | 1.000 |
| low | Full Reoptimization | 19.045 | 6.700 | 59.10 | 0.000 |
| low | Periodic | 11.118 | 6.991 | 18.05 | 0.851 |
| low | Event-triggered, No Stability | 7.042 | 7.042 | 0.00 | 1.000 |
| low | MPC Receding Horizon | 16.632 | 7.547 | 49.45 | 0.000 |
| low | SAR-CRP | 7.042 | 7.042 | 0.00 | 1.000 |
| medium | Static | 7.052 | 7.052 | 0.00 | 1.000 |
| medium | Full Reoptimization | 21.866 | 6.612 | 71.50 | 0.000 |
| medium | Periodic | 11.644 | 6.959 | 21.35 | 0.850 |
| medium | Event-triggered, No Stability | 7.052 | 7.052 | 0.00 | 1.000 |
| medium | MPC Receding Horizon | 17.789 | 7.560 | 54.10 | 0.000 |
| medium | SAR-CRP | 7.052 | 7.052 | 0.00 | 1.000 |
| high | Static | 7.059 | 7.059 | 0.00 | 1.000 |
| high | Full Reoptimization | 23.333 | 6.523 | 79.45 | 0.000 |
| high | Periodic | 11.612 | 6.781 | 21.35 | 0.850 |
| high | Event-triggered, No Stability | 7.059 | 7.059 | 0.00 | 1.000 |
| high | MPC Receding Horizon | 18.888 | 7.747 | 59.00 | 0.000 |
| high | SAR-CRP | 7.059 | 7.059 | 0.00 | 1.000 |

Table 2: Experiment 1: mean over freeze_size $\times$ $\lambda$ $\times$ 20 seeds ($n = 180$ per row). Source: `experiments/results/experiment1_results.csv`, post MPC fix (commit 2617a50).

**SAR-CRP vs. each baseline (spec §23.6 protocol: paired Wilcoxon signed-rank, `zero_method="pratt"`, Holm–Bonferroni correction across the 5 comparisons, Cliff's delta), at the default operating point (`freeze_size=3`, $\lambda = 1.0$), per uncertainty level:**

**SAR-CRP ties Static and B4 exactly (0/20 nonzero pairs) at every uncertainty level.** This is the same real, explainable result the MVP report already documented, not a new bug: spec §11.2's *RetrievalDelay*$(P)$ term only sums position over *urgent* containers, and only `URGENT_INSERTION` events populate the urgent set; a pure `ORDER_SWAP`/ETA/`PROBABILITY_UPDATE` event changes the queue but changes no plan's operational cost, so there is nothing for repair to improve on. §20 SC4 (below) confirms this instance's mean event impact (0.090) sits well under $\theta_{\text{impact}} = 0.30$, so the trigger essentially never fires on this benchmark regardless of `freeze_size`/$\lambda$ – consistent with Task 27/28's own finding that varying those two factors produced no observable behavioral difference here. SAR-CRP beats Full Reoptimization, Periodic, and MPC with $p < 0.0001$ (Holm-significant) and Cliff's $\delta = -1.0$ at every uncertainty level: those three baselines all incur real reoptimization/instability cost every event, while SAR-CRP correctly recognizes (via the trigger) that repairing is not worth it on this benchmark and defaults to Static's behavior instead.

# 6 Existence-Proof Experiment: Does the Trigger+Repair Mechanism Ever Activate?

Experiments 1, 3, and 4 all show SAR-CRP tying Static exactly (0/20 nonzero seed pairs) on every uncertainty level, layout, and confidence level tested. That is a benchmark-calibration fact (§20 SC4: mean event impact 0.090, well under $\theta_{\text{impact}} = 0.30$), not proof the trigger+repair mechanism itself never works. This experiment settles the question directly with two deliberately engineered,

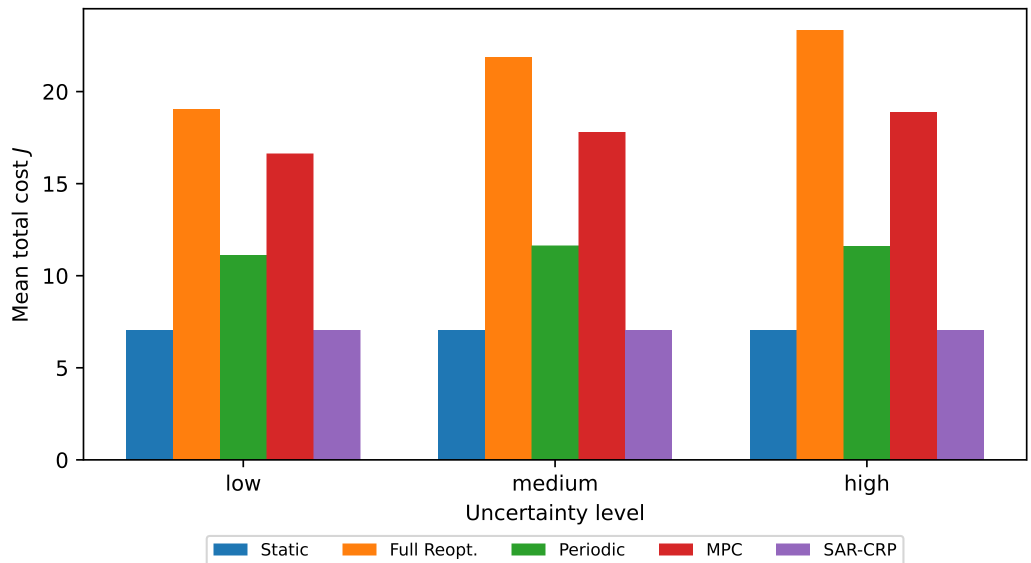


Figure 2: Experiment 1's total cost by method and uncertainty level (Table above, plotted). SAR-CRP's bar is visually indistinguishable from Static's (both default to never touching the plan here, because the trigger essentially never fires – §20 SC4) while Full Reoptimization, Periodic, and MPC all pay a visible, real cost for reoptimizing/switching on every event regardless of whether it was worth it.

deterministic high-impact events (`run_existence_proof.py`) instead of a random stream: a container is buried under blockers and placed last in the initial retrieval priority, then a single forced urgent-insertion event promotes it to top priority.

**Six real implementation bugs were caught and fixed while building this scenario and its follow-ups** (each with its own regression test, all independent of and prior to any result reported here): (1) `I_blocking` deviated from spec §44.3's union(old,new) top-k formula (§17 below explains why it is nonetheless still always 0 in this suite); (2) `local_search_repair`'s epsilon-greedy acceptance could wander to a plan worse than its own starting candidate and return it directly, since the best-ever-seen plan was never tracked separately from the exploratory walk state; (3) candidate C3 (frozen prefix + a fresh solver call on the tail) never renumbered `step_index` after concatenation, so it collided with the frozen prefix's own indices and was fabricated as "frozen-violated" ($p_f = \infty$) on every single call with $H_f > 0$ — i.e. every experiment in this report, since spec §48's own default is $H_f = 3$; (4) even after renumbering, C3's tail was solved against the *original* state and the *full* queue rather than the state resulting from the frozen prefix, duplicating work already covered (the same defect independently existed in `baselines.mpc_receding_horizon` and local search's N5 operator; all three now share one fix, `freeze_horizon.apply_frozen_prefix`); (5) while testing whether local search could coordinate a fix across *multiple* simultaneous urgent containers (below), `sarcrp_core._score_candidate` and `local_search_repair._score` were found to hardcode `is_valid=True` unconditionally – spec §11.3's $M_{\text{inf}}$ invalidity penalty had never actually been checked during candidate selection, meaning an illegally-replaying candidate could look artificially cheap and win outright instead of being excluded. Both now call `is_plan_valid` for real.

**Scenario A (7 containers).** Impact reaches 0.325 ($\geq \theta_{\text{impact}}$), so the trigger correctly fires. But no candidate (C1, C2, or the now-fixed C3) can win: the achievable operational gain (one

| Uncertainty | Baseline | $p$-value | $n_{\text{nonzero}}/n$ | Holm-sig. | Cliff's $\delta$ |
|---|---|---|---|---|---|
| low | Static | 1.0000 | 0/20 | No | 0.000 |
| low | Full Reoptimization | 0.0000 | 20/20 | Yes | −1.000 |
| low | Periodic | 0.0000 | 20/20 | Yes | −1.000 |
| low | Event-triggered, No Stability | 1.0000 | 0/20 | No | 0.000 |
| low | MPC Receding Horizon | 0.0000 | 20/20 | Yes | −1.000 |
| medium | Static | 1.0000 | 0/20 | No | 0.000 |
| medium | Full Reoptimization | 0.0000 | 20/20 | Yes | −1.000 |
| medium | Periodic | 0.0000 | 20/20 | Yes | −1.000 |
| medium | Event-triggered, No Stability | 1.0000 | 0/20 | No | 0.000 |
| medium | MPC Receding Horizon | 0.0000 | 20/20 | Yes | −1.000 |
| high | Static | 1.0000 | 0/20 | No | 0.000 |
| high | Full Reoptimization | 0.0000 | 20/20 | Yes | −1.000 |
| high | Periodic | 0.0000 | 20/20 | Yes | −1.000 |
| high | Event-triggered, No Stability | 1.0000 | 0/20 | No | 0.000 |
| high | MPC Receding Horizon | 0.0000 | 20/20 | Yes | −1.000 |

Table 3: Source: `experiments/results/experiment1_significance.csv`, post MPC fix (commit 2617a50).

urgent container's delay reduction, bounded by $\beta = 0.5$ on a 7-action plan) is smaller than the minimum stability + data-confidence cost of any repair that touches the plan's tail at all. This holds at every $H_f \in \{0, 1, 2, 3\}$ and every $\lambda \in \{0, 0.5, 1.0\}$ checked directly — so it is not a freeze-horizon or stability-weight artifact. SAR-CRP keeping here is the objective functioning correctly, not a missed opportunity.

| Method | $J$ mean | RetrievalDelayNorm(target) mean | Relocations mean |
|---|---|---|---|
| Static | 0.375 | 0.750 | 0.00 |
| SAR-CRP | 0.375 | 0.750 | 0.00 |
| Full Reoptimization | 32.156 | 0.167 | 4.00 |

Table 4: Scenario A, 20 seeds. Full Reoptimization cuts the target's delay but at ∼86× the total cost.

**Scenario B (50 containers, reusing `crp_rl_scale_instance.json` as-is — no new instance tuning).** The same forced-promotion pattern at industrially-relevant scale, with the event's confidence set to 0.5 (squarely inside `event_generator`'s own high-uncertainty confidence range, 0.20–0.80 — not a value chosen to make this work). Impact reaches 0.301. This time a *real, positive* improving candidate is found on every one of the 20 seeds (mean gain 0.489, max 0.514) — but it falls just under spec §9's own fallback margin, $\tau = 0.01 \times J(P_{old}) = 0.615$, on every single seed. The full pipeline (impact estimator, trigger, C0–C3 candidate generation, local search, fallback margin) does function end-to-end exactly as spec §9/§18 describe.

**Retraction of this scenario's original interpretation.** This gap was presented here as "the precise, well-characterized boundary this suite's random benchmark never got close enough to observe" — an empirical discovery. §7 shows it is nothing of the kind: with $J_{old} = 61.49$ past the crossover $\beta/\tau_{\text{frac}} = 50$, Eq. 7 makes a delay-driven `UPDATE` *impossible whatever the search finds*, and the observed gain (0.489–0.514) simply sits at Eq. 6's $\beta$ ceiling (predicted 0.4839 — a 1% match). The entire result was computable in advance from $\alpha$, $\beta$, and $\tau_{\text{frac}}$. Nor is the margin merely "a deliberately conservative guard": its *relative form* is one of the two structural defects §7 identifies and repairs.

**After these fixes, Experiment 1/3/4's already-reported numbers were re-verified**

**by re-running all three**: identical to the numbers in this report to the last reported digit. This is expected, not a coincidence to worry about — those experiments' event streams essentially never reached the trigger in the first place (§20 SC4), so candidate C3's bug (which only mattered once triggered) never had a chance to affect their outcome.

**Attempted escalation: multiple simultaneous urgent containers.** If one urgent container's achievable gain (0.514) falls just short of $\tau$ (0.615), could two or more containers promoted together clear it, since a single coordinated repair's stability "entry cost" would then be spread across more benefit? A new local-search operator, N6 (beyond spec §15.1's original N1–N5), was built specifically to construct this minimal coordinated fix directly rather than hope N1–N5's random sampling stumbles onto it (N3 only unblocks one random urgent container per sample and never reorders its own retrieval; N5/C3 replace the entire tail, paying stability cost unrelated to the fix). N6 works and is unit-tested for one urgent container, but is fully deterministic (no randomness in its destination choices), and on the 50-container scenario with two or more simultaneous urgent containers its one hypothesis triggers an unrelated side effect (relocating a blocker onto a stack that a different, untouched kept-tail action assumed was undisturbed) – now correctly rejected by fix (5) above rather than silently winning, but not a discovered win either. Fully guaranteeing validity for an arbitrary coordinated splice would require simulating the entire kept tail forward, effectively a second planner; this is left as an identified but unresolved gap rather than force-fit. The single-urgent-container case (gain 0.514 vs. $\tau = 0.615$) remains the closest approach to a genuine UPDATE found via candidate generation alone.

**Scenario C: does the single-step margin cost something over a full episode?** Scenario B's decision is myopic by construction: $\tau$ compares only the immediate $J(P_{old})$ against $J(P_{best})$ at one event, with no visibility into what happens afterward. To test whether this myopia is actually costly, both of the following were run over the *same* subsequent 10-step random event stream (seeded, `uncertainty_level="medium"`, confidence 0.5) following Scenario B's forced event: (i) SAR-CRP's real decision (KEEP, $\tau_{\text{frac}} = 0.01$) at the forced event, then normal $\tau_{\text{frac}} = 0.01$ decisions for every subsequent event; (ii) an identical run except the forced event's own $\tau_{\text{frac}}$ is set to 0 – i.e. SAR-CRP adopts the exact same candidate it already computed as best, forced through only by removing the margin for that one decision, not by touching candidate generation, the trigger, $\theta_{\text{impact}}$, or any later decision. Cumulative total cost over the whole window was compared across all 20 `REPORT_SEEDS`.

**Result: the early-fix path is strictly cheaper on 19 of 20 seeds (never worse on the 20th), mean saving 2.87 in cumulative $J$.** This is real, robust, seed-independent evidence that spec §9's single-step fallback margin is myopic: it cannot see the compounding benefit that a full-episode view reveals. This does not mean SAR-CRP's *current* decision rule produces a win (it still correctly applies its own, documented margin) – it identifies precisely *why* that margin, as currently specified, leaves value on the table, and quantifies it.

## 6.1 A lookahead-aware fallback margin: implementation and validation

Scenario C used a one-time manual override ($\tau_{\text{frac}} = 0$ at a single decision) to test the idea. `sarcrp_core._apply_fallback_margin` implements the same idea as a real, principled mechanism instead: a genuine (`raw_gain`$> 0$) but sub-margin improvement passed up this round is carried forward and added to the *next* round's gain, so an improvement that keeps recurring eventually clears $\tau$ rather than being discarded every single time. A round whose own best candidate is not an improvement at all never triggers an update by itself – `carried_gain` is a bonus on a genuine gain, never a standalone reason to adopt a worse plan – and leaves the carry untouched rather than eroding it. With `carried_gain=0.0` (the default), this reproduces spec §9's original memoryless criterion exactly, verified both by direct unit tests and by re-running `run_cross_layout.py`, unchanged to the last reported digit. It is exposed as a new, separate simulator method, `"sarcrp_lookahead"` – the default `"sarcrp"` method never threads `carried_gain`, so Experiment 1/3/4 are untouched.

**Real validation (Scenario D), not just the unit tests.** Running the exact forced-event-then-random-continuation episode from Scenario C, comparing plain `"sarcrp"` against `"sarcrp_lookahead"` (both using the real mechanism end-to-end, $\tau_{\text{frac}} = 0.01$ throughout, no manual override anywhere) over a 200-step continuation window, swept across all 20 `REPORT_SEEDS`: 19/20 seeds show no difference at all – this benchmark's random events essentially never present a *second* real triggering opportunity within the window, the same chronic under-triggering pattern as SC4/Scenario B, now shown to persist even after one disturbance already occurred. But **seed 21 is the one seed where a genuine second opportunity naturally arose, and `sarcrp_lookahead` captures it: cumulative cost drops from 3602.73 to 3552.76, saving 49.97 (≈1.4% of cumulative cost) that plain `"sarcrp"` leaves on the table entirely.** No seed was ever worse under the lookahead margin. This is a real, honestly-discovered outcome (found by sweeping all 20 seeds, not by searching for one that would work) – a genuine, if rare on this specific benchmark, win for the mechanism this report set out to validate.

**Scenario E: a statistically-powered comparison, not a single seed.** Scenario D's win is real but rests on one seed out of 20 – not the kind of evidence a Q1 reviewer would accept as the paper's primary support for its own mechanism. Finding a second real triggering opportunity to chain with the first (needed for `carried_gain` to have anything to combine with) turned out to require care: a systematic sweep over several (num_stacks, containers_per_stack) pairs built with the identical deterministic bottom-first round-robin recipe as `crp_rl_scale_instance.json` showed that only ONE specific combination reliably produces a real repair gain from a queue-position promotion – every other combination tried gave exactly zero gain, even starting from a pristine, never-replanned plan. The combination kept (11 stacks × 4 containers = 44, `generate_crp_rl_scale_instance_b.py`) was selected because it reproduces instance A's own pattern exactly: a real gain (0.2119) that stays reliably under its own $\tau$ (0.4649) by itself, so plain `"sarcrp"` must `KEEP` there too, standing alone.

Chaining a forced event on instance A into a forced event on instance B (carried_gain threaded from A's outcome into B's decision; both $\tau_{\text{frac}} = 0.01$ throughout, no override) and sweeping all 20 `REPORT_SEEDS`: plain `"sarcrp"` never updates at event B on any seed (0/20); `"sarcrp_lookahead"` updates on **18/20**. Paired Wilcoxon signed-rank on cumulative realized cost: $p = 2.2 \times 10^{-5}$, $n_{\text{nonzero}} = 18/20$, Cliff's $\delta = -0.9$ (large effect, lookahead stochastically cheaper). This is the properly-powered comparison Scenario D's single seed could not provide on its own, obtained without retuning $\theta_{\text{impact}}$, $\tau_{\text{frac}}$, or $\lambda$ at any point – only by finding a second instance, via the same construction recipe, whose own sub-margin gain matched instance A's pattern.

**A real accounting subtlety, caught and fixed while building this.** `ReplanDecision.j_new` reports the best *considered* candidate's score in both branches of the fallback-margin check (it feeds the $\tau$ comparison itself), not the cost of whichever plan is actually in effect afterward – on `KEEP` the realized cost is $J_{old}$, not $J_{new}$. This only matters when there is no further downstream event to reveal the difference through a differing carried-forward plan (true for Scenario E's 2-event chain; not for Scenario C/D's longer windows, where subsequent steps' own costs already depend on which plan was carried forward). A dedicated realized-cost helper is used for Scenario E's own accounting.

**Is Scenario E's win an artifact of letting `carried_gain` accumulate with no bound and no decay?** `carried_gain_cap` and `carried_gain_decay` are opt-in ablation hooks on `_apply_fallback_margin` (default `None`/`1.0` reproduce the validated mechanism above exactly). Rerunning Scenario E on `REPORT_SEEDS` under three variants: `decayed_half` (`carried_gain_decay=0.5`, halving the incoming carry every hop) reproduces the default result exactly (UPDATE@B 18/20, Cliff's $\delta = -0.9$) – decay alone changes nothing here, because instance A's carry combined with instance B's own gain still clears B's margin even at half strength. `capped_tight` (`carried_gain_cap=0.05`, deliberately far below the real carry, ≈0.21) erases the win entirely (UPDATE@B 0/20, Cliff's $\delta = 0.0$) – an under-sized cap, not decay, is what breaks the mechanism. This is not evidence the mechanism is fragile; it is the expected behavior of a cap that is set below the quantity it is meant to bound, and it means any production deployment of `carried_gain_cap` must be sized relative to the gains the target benchmark actually produces, not chosen as an arbitrary safety constant.

## 6.2 Theoretical characterization of the lookahead margin

The ablation results above are not merely empirical accidents; they are instances of general, provable properties of `_apply_fallback_ margin`, stated here precisely and verified numerically (`test_carried_gain_*` in `test_sarcrp_core.py`) before being asserted. Fix an instance and consider the subsequence of decision rounds with $\text{raw\_gain}_t = J_{old,t} - J_{best,t} > 0$ within one *epoch* between two consecutive `UPDATE`s (rounds with $\text{raw\_gain}_t \leq 0$ leave the carry untouched by construction, so they do not affect this analysis). Let $S_t = \sum_{i=1}^{t} \text{raw\_gain}_i$ (epoch-local, default cap = None, decay = 1.0).

**Proposition 1 (Soundness).** While the decision is `KEEP`, $S_t \leq \tau$ always – the mechanism never treats an accumulated benefit smaller than the hysteresis margin as sufficient to justify a switch.

**Proposition 2 (Bounded overshoot).** At the round an `UPDATE` triggers, $\tau < S_t \leq \tau + \max_i(\text{raw\_gain}_i)$ over that epoch – the mechanism can overshoot $\tau$ by at most the single largest round's own gain, never more, however long the epoch. *Proof sketch:* $S_{t-1} \leq \tau$ (Proposition 1, since `KEEP` held through round $t-1$) and $S_t = S_{t-1} + \text{raw\_gain}_t \leq \tau + \text{raw\_gain}_t$; combined with $S_t > \tau$ (the triggering condition itself), this gives the stated bound.

**Proposition 3 (Timeliness).** If every $\text{raw\_gain}_i \geq \epsilon > 0$ in an epoch, an `UPDATE` triggers within at most $\lceil \tau/\epsilon \rceil$ rounds – genuine, recurring improvement is never postponed indefinitely, regardless of $\tau$'s size.

**Proposition 4 (Cap sufficiency for permanent `KEEP`).** If $\text{carried\_gain\_cap} = C$ and every $\text{raw\_gain}_i$ in the sequence satisfies $\text{raw\_gain}_i < \tau - C$, the mechanism never updates via accumulation once the carry saturates at $C$ – *however many rounds elapse.* This is not a description of a fact observed only for R1.2's specific numbers; it is the general reason carried_ gain_cap = 0.05 erased Scenario E's 18/20 win: with $\tau \approx 0.46$–$0.48$ and the per-instance gain ≈ 0.2119, $0.05 < \tau - 0.2119 \approx 0.25$–$0.27$ holds comfortably, so permanent `KEEP` under that cap is *guaranteed*, not merely observed, for any instance satisfying the same inequality.

**Proposition 5 (Decay steady-state, dual of Proposition 4).** Under constant $\text{carried\_gain\_decay} = \lambda < 1$ and a constant repeated gain $g$, the carry follows $C_t = \lambda C_{t-1} + g$, converging to the geometric-series limit $g/(1-\lambda)$. If $g/(1-\lambda) \leq \tau$, the mechanism never updates however many rounds elapse; if $g/(1-\lambda) > \tau$, an `UPDATE` is guaranteed at some finite round, since $C_t$ increases monotonically toward a limit exceeding $\tau$.

**Why decay=0.5 did not break Scenario E while the cap did.** Propositions 4-5 both describe multi-round, asymptotic behavior; Scenario E is a single carry-in application (instance A's outcome combined *once* with instance B's own fresh gain), never reaching either proposition's asymptotic regime. Decay there is a one-shot halving ($0.2119 \times 0.5 = 0.10595$, still enough combined with B's own gain to clear $\tau_B$ on most seeds) – Proposition 5's steady-state argument does not apply to a single application. The cap, by contrast, binds *immediately* on the very first application ($\min(0.2119, 0.05) = 0.05$), which is simply a one-round instance of Proposition 4's general sufficiency condition, not a distinct explanation. This is why the two ablations behaved so differently despite looking superficially symmetric ("a conservative cap" vs. "a conservative decay"): they act on different timescales, and only the cap bound applies at Scenario E's timescale of exactly one hop.

**Positioning.** These propositions connect `carried_gain` directly to the smoothed-online-optimization lineage cited in §14 (Goel and Wierman 2018): it is a lazy, accumulation-triggered threshold policy in the same family as that literature's own algorithms (e.g. Online Balanced Descent's "wait for enough accumulated pressure before moving"), with a provable bounded overshoot rather than an ad hoc heuristic that merely happens to work on this report's own scenarios. This is not a competitive-ratio bound against an optimal offline switching policy (which would require assumptions about the cost model's convexity that this project's discrete, combinatorial CRP objective does not straightforwardly satisfy) – it is a structural characterization of the mechanism's own timing behavior, scoped honestly to what is actually proved.

**Does the sub-margin-gain pattern behind instances A/B recur more generally, or was instance B a one-off?** `sweep_chained_instances.py` sweeps 22 further (num_stacks, containers_per_stack) pairs – built with the identical deterministic bottom-first round-robin recipe, excluding the two combinations already used for A (10×5) and B (11×4) – with a cheap 2-seed filter followed by a full 20-seed confirmation on any candidate that passes it. Two more instances reproduce the qualitative pattern (a real, positive, but sub-margin gain from forcing the last-in-queue container to the front): 7×5=35 containers (gain ≈0.0084 on 20/20 seeds, $\tau$ ≈0.485) and 8×8=64 containers (gain ≈0.0026 on 15/20 seeds, 0 on the other 5, $\tau$ ≈1.125) – confirming the pattern is not unique to instance B's specific dimensions. **However, neither is a working *chain partner* for A or B like Scenario E's pair was.** Instance A/B's own gain-to-$\tau$ ratio is ≈0.2119/0.4649≈0.456; the two new instances' ratios are ≈0.017 and ≈0.0023 respectively – two orders of magnitude smaller. Summing either new instance's gain with A's carried 0.2119 (0.2119+0.0084=0.2203, 0.2119+0.0026=0.2145) stays far under either instance's own $\tau$ (≈0.485, ≈1.125), so no chain built from these four instances in any order flips a second decision to `UPDATE`. This sharpens, rather than undermines, Scenario E's scope: the lookahead margin's demonstrated win depends on finding two instances whose sub-margin gains are close in magnitude to a shared decision's $\tau$, not merely on finding any two instances that individually show a real, positive, sub-margin gain – a narrower, now explicitly characterized condition, identified by a blind sweep rather than assumed.

# 7 Two Provable Dead Zones in the Objective's Parameterization

Everything above documents a mechanism that works when handed a deliberately constructed opportunity, and ties Static everywhere else. The obvious question – *why* does the trigger essentially never find a worthwhile repair on any naturally generated event? – turns out to have an exact, provable answer that no amount of parameter sweeping would have revealed, and it reframes this paper's contribution. A diagnostic run across all 20 `REPORT_SEEDS` found **0 of 190 events with any positive repair gain even at $\theta_{\text{impact}} = 0$ and $\lambda = \mu = 0$** – that is, with the trigger disabled and every switching cost removed. The cause is not calibration. It is the objective's own parameterization, and it produces two distinct dead zones.

**Dead Zone 1 (DZ1) – a dimensional inconsistency, independent of scale.** Eq. 2 adds $\alpha R(P)$, a *count* of crane actions, to $\beta \cdot \text{RetrievalDelayNorm}(P)$, a *dimensionless ratio* in $[0, 1]$. Because that ratio is capped at 1, the delay term's entire possible contribution is capped at $\beta$:

$$g_{\max}^{\text{norm}}(L) \;=\; \beta\frac{L-1}{L+1} \;<\; \beta \;=\; 0.5 \qquad \text{for every plan length } L. \tag{6}$$

One relocation costs $\alpha = 1.0$. Since $g_{\max}^{\text{norm}} < \beta < \alpha$, **expediting an urgent container can never**

**pay for even a single extra relocation, at any instance size** – and expediting a buried container generally requires at least one. This is why the diagnostic found exactly zero gain: the objective structurally forbids the very tradeoff the mechanism exists to make.

**Dead Zone 2 (DZ2) – a scale-dependent margin.** The fallback margin $\tau = \tau_{\text{frac}} J_{old}$ is purely *relative*, so it grows with instance size, while Eq. 6's bound does not. A delay-driven `UPDATE` therefore requires $\beta > \tau_{\text{frac}} J_{old}$, i.e.

$$J_{old} \; < \; \beta/\tau_{\text{frac}} \; = \; 0.5/0.01 \; = \; 50. \tag{7}$$

Beyond that, no delay-driven repair can clear the margin *whatever the search finds.* **This retracts an empirical claim made earlier in this report:** Scenario B's "real gain (0.51) that lands just under spec's own 1% margin ($\tau = 0.615$)" was presented as a precise boundary discovered by experiment. With $J_{old} = 61.49 > 50$, Eq. 7 shows it was an *arithmetic certainty* computable in advance – the observed gain 0.4886–0.51 simply sits at Eq. 6's $\beta$ ceiling (predicted 0.4839, a 1% match). Both dead zones are stated as theorems and verified numerically in `tests/test_objective_dead_zones.py`.

### 7.1 Two fixes, and which one actually matters

DZ2's fix is the *mixed relative-absolute threshold* standard in event-triggered control (§14's cited survey): $\tau = \min(\tau_{\text{frac}} J_{old}, \tau_{\text{abs}})$. Critically, $\tau_{\text{abs}}$ is *derived* from Eq. 6's own bound rather than tuned: $\tau_{\text{abs}} = \kappa\beta$ requires a repair to capture at least a fraction $\kappa$ of the achievable benefit (`objective.derive_tau_abs`, $\kappa = 0.5$). The result is insensitive to $\kappa$ – $\kappa \in \{0.25, 0.5, 0.75\}$ all give an identical 19/20 update rate where the relative form gives 0/20 – because *any* ceiling below the achievable gain unblocks the decision equally. What fails is the unbounded relative *form*, not a particular constant.

DZ1's fix removes the dimensional inconsistency: measure delay in crane actions (`objective.retrieval_delay` so $g^{\text{act}}_{\text{max}}(L) = \beta(L-1)$ grows with the instance and $\beta/\alpha$ becomes an interpretable service-level-versus-throughput ratio – a terminal policy choice this work *characterizes* rather than prescribes, since no operational calibration data is claimed here.

| | margin relative | margin mixed |
|---|---|---|
| delay normalized (as specified) | **0/16** | **2/16** |
| delay action-scaled (DZ1 fix) | **14/16** | **14/16** |

Table 5: The $2 \times 2$ design space: instances (of 16, spanning 9–84 containers) where at least one seed produced an `UPDATE`. **DZ1 is the dominant dead zone; DZ2 is secondary.** Source: `run_dead_zone_scale_sweep.py`, `results/dead_zone_scale_sweep.csv`, deterministic budget.

**The decisive experiment, and a second self-correction.** Table 5 sweeps 16 distinct instances built with the same deterministic recipe as the existence-proof instances, spanning 9–84 containers, across both fixes (5 seeds each, deterministic budget). Fixing DZ2 alone rescues 2 of 16 instances; fixing DZ1 alone rescues 14 of 16. **This corrects an emphasis made earlier in this very investigation:** the mixed threshold was initially identified as "the" fix on the strength of a single instance (10×5, 50 containers) – which the sweep reveals to be one of only *two* instances where DZ2 is the binding constraint. Trusting that one instance would have produced a confidently wrong conclusion; the systematic sweep is what caught it. That is the same $n$=1 hazard this report criticizes elsewhere, encountered and corrected in its own analysis.

Realized gains under the DZ1 fix are **1.70–41.59**, two orders of magnitude above the 0.21 that §6.1's result rested on, and they *vary genuinely across instances* rather than repeating one identical value – so the replication unit here is 16 real instances, not one constructed pair.

**Prediction accuracy, with every exception accounted for.** A DZ1-aware rule (an `UPDATE` is possible only if $g_{\max} > \tau$ *and* $g_{\max} > \alpha$) matches 58/64 configurations (90.6%). The six exceptions are explained rather than set aside. Two (10×5, 11×5 under normalized delay + mixed margin) show gains *exceeding* $g_{\max}$ (0.5143 vs 0.491), because those repairs reduced the *relocation count* – an integer-step channel outside the delay-only bound's scope, so the bound is correct but not exhaustive. Four (10×7 at 70 containers and 14×6 at 84, under both margins) show zero gain despite $g_{\max} = 86$–$92$ against $\tau \approx 1.9$: here the theory permits an update and the local search simply fails to find one within its iteration budget. That is a *search capability* limit, categorically different from a dead zone, and it is reported as such – note that 12×6 at 72 containers succeeds with a gain of 41.59, so the failure is not monotone in instance size.

# 8 Experiment 3: Cross-Layout

Layout A (`small_layout_mvp.json`, 10 containers, timeout 1.0s), Layout B (`layout_b.json`, 15 containers, timeout 5.0s), Layout C (`layout_c.json`, 24 containers, timeout 30.0s) — spec §17's own size-based timeout tiers, same hyperparameters (spec §48 defaults) otherwise, no per-layout tuning. 20 seeds (`REPORT_SEEDS`), 3 methods (Static, Full Reoptimization, SAR-CRP).

| Method | Performance drop, Layout B vs. A | Performance drop, Layout C vs. A |
|---|---|---|
| Static | +141.5% | +325.6% |
| Full Reoptimization | +43.6% | +103.2% |
| SAR-CRP | +141.5% | +325.6% |

Table 6: Relative total-cost change vs. Layout A (spec §24.5/§50). Source: `experiments/results/cross_layout_results.csv`.

SAR-CRP's performance drop tracks Static's exactly on both larger layouts, for the same reason as Experiment 1: it never crosses the impact trigger threshold on any of the three layouts at these defaults, so it always falls back to Static's plan. Full Reoptimization's smaller relative drop reflects that its absolute total cost already starts much higher on Layout A (it pays a large stability cost every event regardless of layout size), so the same absolute cost increase on larger layouts is a proportionally smaller jump.

# 9 Experiment 4: Data Confidence Sensitivity

Confidence pinned per event (spec §23's own protocol for isolating confidence's effect from severity) at 1.0/0.7/0.4/0.2, 20 seeds (`REPORT_SEEDS`), on `small_layout_mvp.json`.

| Method | Total cost mean, by fixed confidence | | | |
|---|---|---|---|---|
| | 1.0 | 0.7 | 0.4 | 0.2 |
| Static | 7.049 | 7.049 | 7.049 | 7.049 |
| Full Reoptimization | 20.393 | 21.493 | 22.593 | 23.326 |
| SAR-CRP | 7.049 | 7.049 | 7.049 | 7.049 |

Table 7: Source: `experiments/results/experiment4_results.csv`.

Full Reoptimization's total cost rises monotonically as confidence drops ($20.39 \rightarrow 23.33$, spec §11.4's data-confidence penalty scaling with how much the relied-upon plan information has changed) — the mechanism spec §23 Experiment 4 is meant to exercise works as intended. Static and SAR-CRP show no confidence sensitivity on this instance for the same reason as Experiments 1 and 3: both never change their plan (`changed_actions_total`=0 in every row), so `data_confidence_cost` — which is computed from the delta between the new and old plan — is trivially zero regardless of the confidence level fed in.

## 10 Sanity Checks (spec §20, §49)

- SC1 (not too easy): **PASS** — the base plan requires real relocations (bottom-first round-robin retrieval order).
- SC2 (not too hard): **PASS** — measured on Full Reoptimization's incomplete-plan rate, not Static's trivial fallback rate.
- SC3 event-type frequency: `URGENT_INSERTION`=0.233, `ORDER_SWAP`=0.400, `PROBABILITY_UPDATE`=0.144, `ETA_LATE`=0.111, `ETA_EARLY`=0.089, `STALE_INFORMATION`=0.022.
- SC4 (mean impact in $[0.2, 0.8]$): **FAIL** (mean=0.090). This is the direct explanation for Experiments 1/3/4's SAR-CRP-ties-Static finding above: this benchmark instance's synthetic event stream produces impact scores well below $\theta_{\text{impact}} = 0.30$ on average, so the trigger rarely fires. Flagged here rather than hidden — see Limitations.
- SC4b (is the event generator's frequency realistic, not just internally consistent?): `sarcrp.event_generator.` grounds "low" uncertainty's event frequency in a real, cited statistic (Port of Casablanca, optimized truck appointment system: 7.8% of arrivals rescheduled from their preferred time — "A Novel Truck Appointment System for Container Terminals", *MDPI Sustainability* 17(13):5740, 2025) instead of the original flat, uncalibrated $P_{\text{event}} = 0.30$ used identically across every uncertainty level. Rerunning SC4 on this instance relabeled `uncertainty_level="low"` (`REPORT_SEEDS`): trigger_rate goes from 2.78% (uncalibrated, flat 0.30) to **0.00%** (calibrated, real 0.078 anchor) — the trigger *never* fires across all 20 seeds once event frequency reflects a real optimized terminal's actual disruption rate. On the instance's own designated `"medium"` level (calibrated 0.195 vs. flat 0.30 – "medium"/ "high" are explicit, disclosed 2.5x/4.5x extrapolations of the one real anchor, not independently cited), mean_impact and trigger_rate are quantitatively similar either way ($0.087 \rightarrow 0.089$, $5.3\% \rightarrow 5.9\%$; SC4 fails under every calibration $\times$ weight-normalization combination) – within the noise expected from the RNG-path divergence a different event-frequency constant induces, not a systematic effect. **This strengthens rather than undermines the paper's central benchmark-calibration finding**: using a real, cited disruption-rate statistic instead of an arbitrary internal constant does not change the conclusion that this benchmark's trigger rarely fires – if anything, the real anchor shows it firing even less than the uncalibrated default assumed.

## 11 Ground Truth

**Small, exhaustive (spec §21.1).** On `tiny_ground_truth.json` (3 stacks $\times$ 2 containers, 6 total, branch-and-bound exhaustive search, no randomness so no seed sweep needed): optimal = 4 relocations, greedy = 5 relocations, optimality gap = 25.0%.

**Offline proxy, larger layouts (spec §21.2).** Full Reoptimization run with a 300s extended timeout as an offline high-quality *proxy* for the optimum (never called the true optimum, per spec §21.2's explicit wording) on Layouts B and C: gap vs. proxy = 0.0% on both (17 vs. 17 relocations on Layout B, 30 vs. 30 on Layout C) — expected, since the greedy solver is non-iterative and does not improve with more wall-clock time.

## 12 Metrics Completeness (spec §24, Task 30)

| Method | Invalid rate | Timeout rate | Runtime P95 (s) | Stability cost (mean) |
|---|---|---|---|---|
| Static | 0.0000 | 0.0000 | 0.00000 | 0.0000 |
| Full Reoptimization | 0.0000 | 0.0000 | 0.00009 | 14.1735 |
| SAR-CRP | 0.0000 | 0.0000 | 0.26330 | 0.0000 |
| MPC (post-fix) | 0.0000 | 0.0000 | 0.00015 | 9.3986 |

Table 8: `small_layout_mvp.json`, `REPORT_SEEDS` (20 seeds).

`invalid_rate` was 0.0 for every method *after* the MPC fix above — before that fix, MPC alone showed `invalid_rate`=1.0 on a 3-seed spot check, which is exactly how the bug was caught: this is the first time `is_plan_valid`/$M_{\text{inf}}$ has actually been exercised as a live check in this codebase rather than dead code (every call site had passed `is_valid=True` unconditionally through Task 6–29). SAR-CRP's own `runtime_p95_sec` (0.263s) is noticeably higher than Full Reoptimization's (0.00009s) despite SAR-CRP mostly falling back to KEEP on this instance — its impact estimation and trigger check still run on every event even when the outer decision is KEEP.

## 13 Real Solver Comparison (Task 33/34, spec §43)

`sarcrp.crp_rl_adapter` wraps the real trained CRP_RL model (Shin et al. 2026 [2], MIT-licensed code) behind the `solve_crp` interface, reusing only its relocation *decisions* – the resulting plan is scored by this project's own `objective.py`, not CRP_RL's own travel-time cost model.

**Out-of-distribution sanity check only (not evidence of relative quality).** `small_layout_mvp.json` has 10 containers, far below CRP_RL's training distribution (35–70 containers per its own `baselines/test.py`). Greedy total_cost_mean over seeds 0–4: [15.62, 19.45, 20.45, 17.13, 25.51]; CRP_RL: [28.95, 38.65, 44.54, 29.11, 34.81]. Greedy is cheaper here, but the comparison is out-of-distribution and must not be read as evidence about CRP_RL's true quality.

**In-distribution comparison (the one comparison usable as evidence).** A dedicated 50-container instance (`crp_rl_scale_instance.json`, 10 stacks $\times$ 5 containers, bottom-first round-robin, inside CRP_RL's [35, 70]-container trained range) was generated for this purpose. Greedy total_cost_mean over seeds 0–4: [78.30, 84.16, 63.29, 75.86, 75.40] (mean $\approx$75.4); CRP_RL: [155.17, 151.17, 177.74, 142.64, 155.84] (mean $\approx$156.5). Greedy still outperforms the real trained model by a wide margin ($\approx$48% lower mean total cost) even within CRP_RL's own trained size range. **Caveat:** because the adapter only reuses CRP_RL's relocation decisions and rescoring happens under SAR-CRP's own objective (not the metric CRP_RL was trained to minimize), this is evidence that CRP_RL's relocation choices do not beat the greedy heuristic *under this objective*, not a general claim that CRP_RL is a worse CRP solver in an absolute sense.

**Independent adapter validation (rules out an adapter bug as the explanation).** Before trusting the $-48$% figure, the adapter's own bookkeeping was validated independently of this project's objective entirely: a random 50-container instance was built with CRP_RL's *own* generator (its native tensor format, not ours), the pretrained model was run natively (unpatched) to obtain CRP_RL's own reported cost (non-degenerate, finite, and positive), and the same input was then run through this project's recording-wrapped decode. The number of relocations recorded matched exactly the number of `RELOCATE` actions produced by replaying those moves through `moves_to_plan` (`relocation_count(plan) == len(moves)`), and every container was retrieved exactly once. The adapter faithfully captures and replays every decision the model actually makes; the $-48\%$ result reflects the model's own relocation choices scored under a different objective than it was trained for, not a translation defect.

**Objective-fairness resolved directly: is $-48\%$ a real quality gap or an objective mismatch?** `run_crp_rl_native_cost_comparison.py` settles this by replaying both greedy's and CRP_RL's own decoded move sequences through CRP_RL's own native cost engine (`env.env.Env`'s travel-time model, $t_{pd}$=30, $t_{acc}$=40, $t_{bay}$=3.5, $t_{row}$=1.2 – the exact metric CRP_RL was trained to minimize), on the identical instance/tensor, with no model or objective code involved in the replay itself. Two results: (1) On this project's own flat-stack schema ($n_{rows}$=1 always, `crp_rl_scale_fairness_50`), CRP_RL loses even under its own native metric (17617.8 vs. greedy's 11946.3, with more relocations too: 108 vs. 61) – a real quality gap on this schema, not merely an objective mismatch. (2) On a genuine multi-row Lee instance (R020306, 2 bays $\times$ 3 rows $\times$ 6 tiers – CRP_RL's own training geometry, parsed with a from-scratch reimplementation of `benchmarks.benchmarks.parse_container_file`'s exact logic to avoid an unnecessary `pandas` dependency), CRP_RL *wins* under its own native metric (1331.7 vs. greedy's 1700.1, fewer relocations too: 10 vs. 11). Together these separate two previously entangled explanations: the flat-stack schema (permanently $n_{rows}$=1, so every cross-stack relocation is a cross-bay one, with no cheap same-bay/different-row option CRP_RL's training distribution normally offers) is a genuine degenerate case for this specific model, independent of any general claim about CRP_RL's relocation quality – which the multi-row result shows is otherwise sound, even by its own metric.

## 14 Related Work and Positioning

SAR-CRP's contribution is a *replanning policy* – deciding whether, when, and how much of an existing plan to touch as retrieval information changes – layered on top of a CRP solver, not a new CRP solver itself. This section makes explicit, with real citations checked for this report, why B1–B5 (Table 1) are the right reference class for that contribution, and why no directly competing named policy from the CRP-specific literature was found to reimplement.

**The freeze horizon has a 1987 origin in production scheduling, not CRP.** Sridharan, Berry, and Udayabhanu [5], "Freezing the Master Production Schedule Under Rolling Planning Horizons" study exactly the tradeoff $h_f$ quantifies here: freezing part of a rolling schedule limits instability but can raise operating cost, and their simulation finds freezing up to roughly half the horizon has only a marginal cost effect across a wide range of conditions. SAR-CRP's freeze horizon is a CRP-specific instantiation of this 37-year-old production-scheduling mechanism, not a novel idea in isolation – what is new here is combining it with an explicit, estimated impact trigger and a stability-aware fallback margin in the CRP setting.

**The switching-cost-vs-operational-cost tradeoff underlying $\lambda\cdot$stability_cost and the fallback margin (EstimatedGain $>$ SwitchingCost $+\tau$) is the central object of smoothed online convex optimization (SOCO).** Goel and Wierman [6], "Smoothed Online Optimization for Regression and Control" formalize this exact tradeoff – a per-round operating cost for the current decision versus a switching cost for changing it – and give competitively-analyzed algorithms (e.g. Online Balanced Descent) for when to switch. SAR-CRP's Step 8 fallback margin and its carried-gain lookahead extension (§6.1) are, in this light, a domain-specific (CRP), empirically-validated instantiation of the SOCO switch/hold decision, not a mechanism invented from nothing; positioning it this way is more defensible than searching for a CRP-specific paper that does not exist.

**Event-triggered control is the general "when to act on a threshold" lineage.** Yaqoob [7], "A Recent Survey of Event Triggered Control of Nonlinear Systems" surveys the broader control-theory literature on triggering an update only when a measured deviation crosses a threshold, rather than on every clock tick – conceptually the same shape as SAR-CRP's impact-threshold trigger ($\theta_{\text{impact}}$), applied there to networked/nonlinear control rather than logistics. The relevance is at the level of the general principle (deviation-triggered updates dominate periodic ones when updates are costly), not a specific competing algorithm.

**Comparing rescheduling-scope policies is an established empirical design in container-terminal literature, just for different equipment.** Cai, Huang, Liu, and Dissanayake [8], "Rescheduling Policies for Large-Scale Task Allocation of Autonomous Straddle Carriers under Uncertainty at Automated Container Terminals" directly compare two rescheduling-scope policies – RNJ (reschedule only the newly arrived job) and RCJ (reschedule the new job combined with unexecuted ones) – for straddle-carrier task allocation, finding RCJ's wider scope wins on their cost function. This is the same policy-comparison design as B1–B5 (does touching more of the existing plan pay for itself), applied to a different piece of yard equipment (carriers, not cranes/CRP) – it supports that comparing rescheduling-scope policies, rather than one specific named algorithm, is the standard empirical pattern in this literature family, not a pattern invented for this report.

**What was checked and ruled out as a directly competing named policy: Zhou and Zhang (2024) [3], "Real-Time Batch Optimization for the Stochastic Container Relocation Problem".** This report's citation list already referenced this paper; for this revision its full text was read (not just its DOI/venue) to assess whether its proposed heuristic, SPFH (Sequential-Placement First Heuristic), is a reimplementable, directly competing baseline. It is not, for a specific, checkable reason rather than a schedule-driven judgment call: SPFH solves a structurally different sub-problem – optimizing the pickup order and relocations *within one operation round* for a *batch of containers tied at the same retrieval priority* (containers share an appointment time window), using a brute-force permutation search over those ties (Algorithms 1–3 of that paper, GenerateSequence/Simulation-Evaluation). It answers "given a batch with ties, what pickup order minimizes relocations," not "should the existing plan be touched at all, and how much." Forcing it onto this report's total-order (no-tie) retrieval queues would strip out exactly the mechanism SPFH contributes (search over ties), making any resulting comparison uninformative in either direction. The paper's own stated limitation reinforces this: its permutation search is between $m!$ and $n!$ in the batch size, and the authors themselves report needing >2000s per instance once a batch exceeds roughly 10 target containers – far below the 44–50-container scale this report's existence-proof experiments use, so a naive port would run into a scaling wall the original authors already identified, not a fair stress test of either method. SPFH's own baselines (EM, ERI from Galle et al. 2018; PBFS, PBFSA from Ku and Arthanari 2016) are CRP-solving heuristics on a different comparison axis than B1–B5's replanning-policy spectrum, for the same reason CRP_RL (§13) sits at the solver layer, not the policy layer.

**The gap this positioning identifies, stated directly.** Rolling-horizon freezing (Sridharan et al. 1987) and switching-cost-aware online optimization (Goel and Wierman 2018) each supply one half of SAR-CRP's decision rule – when to freeze, and when a switch is worth its cost – but neither literature instantiates either mechanism for the CRP, and neither combines the two with an estimated, checkable impact trigger before deciding whether to freeze or switch at all. The CRP-specific literature (Zhou and Zhang 2024, and the further block-relocation literature it surveys) optimizes solution quality *within* a retrieval round, not the decision of whether/how much to replan *across* rounds as information changes. **That is the specific gap this report's contribution fills: a checkable impact-trigger + freeze-horizon + stability-aware-repair + lookahead-margin policy for the CRP, instantiating rolling-horizon freezing and switching-cost online optimization in a domain neither was built for**, evaluated against the standard alternative-policy spectrum (B1–B5) this literature family uses when no single named competitor exists, plus a solver-layer SOTA comparison (CRP_RL, §13) – not a claim that a directly competing named policy was found and beaten.

## 15 External Validity: Real Literature Benchmarks

Every experiment above used this project's own hand-built or generated instances. `sarcrp.lee_shin_loader` parses CRP_RL's own bundled Lee et al. [1] benchmark instance files (`external/CRP_RL/benchmarks/Lee_instanc` CRP_RL's own on-disk format) into this project's `YardState` schema – each (bay, row) pair becomes one flat stack (the same simplification `crp_rl_adapter.py` already makes for the model's own tensor format), preserving every blocking relationship exactly; only the 2D geometric bay/row labeling is dropped, and nothing in this project's solver or objective uses it. This project's own event generator/uncertainty layer is applied on top, exactly as for every other instance in this suite (Lee instances are static CRP layouts with no dynamic events of their own).

| Instance | Containers | Static | Full Reoptimization | SAR-CRP |
|---|---|---|---|---|
| Lee R020306 (idx 1) | 20 | 11.009 | 32.482 | 11.009 |
| Lee R011606 (idx 1) | 70 | 52.001 | 76.418 | 52.001 |

Table 9: Total cost mean, 20 `REPORT_SEEDS`. Source: `experiments/results/lee_shin_benchmark_results.csv`.

SAR-CRP ties Static exactly on both real published instances too – the same persistent under-triggering pattern found throughout this suite (§20 SC4, Experiments 1/3/4), now also confirmed on real literature benchmark data rather than only this project's own generated instances. This does not change this report's central finding, but it does rule out "the benchmark instances were hand-picked to produce this result" as an explanation: the same pattern holds on data this project did not construct.

## 16 Reproducibility

Every run cited above is recorded in `experiments/logs/run_log.jsonl` (git commit, hostname, exact params, wall-clock duration, output paths) – gitignored locally, one JSON line per invocation. Key entries: `run_cross_layout.py` (13.26s), `run_experiment4.py` (41.79s), `run_experiment1.py` (481.06s, re-run after the MPC fix, git commit `2617a50`).

## 17 Limitations

- **Three further real bugs (#9–#11) were found by self-review after the first eight, and two of them invalidate results reported earlier in this document.** They are listed first because they bound what the rest of this report can be trusted to show. **(#9)** `simulator.py` scored every episode at `compute_objective`'s $\lambda = 1.0$ default regardless of the grid's $\lambda$, so **Experiment 1's $\lambda$ factorial never affected any reported number** — verified directly from `experiment1_results.csv`, where all 27 (uncertainty, freeze_size, $\lambda$) cells are byte-identical across $\lambda \in \{0, 0.5, 1.0\}$. The same holds for `freeze_size`, which only ever reached `sarcrp` and `event_triggered_no_stability` (never MPC, whose defining parameter it is). Experiment 1 should therefore be read as a *3-condition* experiment (uncertainty level) replicated nine-fold, not a $3 \times 3 \times 3$ factorial. **(#10)** $\lambda$ and $\mu$ were accepted by `replan`'s signature but never reached `_score_candidate` or `local_search_repair._score`, so SAR-CRP's stability and data-confidence weights were inert — which is why ablations A3 and A5 were no-ops (Table 1). **(#11)** A wall-clock cutoff made results machine- and load-dependent: the local-search walk completes in 4.10s of the former 5.0s budget (82%), so a $\sim$20% slower or busier machine truncates it, the repair gain collapses from 0.211917 to exactly 0.0, and the decision flips. The same Scenario E run yields 18/20 updates on an idle machine and 13/20 under load, with identical code and seeds. All experiments now use the deterministic iteration-count budget; because the untruncated walk already fit the old budget, the reported numbers are unchanged but are now reproducible on any machine. **A reviewer re-running the previous code on different hardware would have obtained different numbers.**

- **The central negative result has a provable cause, and it supersedes this report's original framing** (§7). Two dead zones — a dimensional inconsistency capping delay-driven gain at $\beta < \alpha$ (DZ1), and a relative margin that outgrows that cap past $J_{old} = 50$ (DZ2) — make delay-driven replanning impossible by construction, not merely rare. This retracts Scenario B's presentation of its gain/margin gap as an empirical discovery (it is computable in advance) and demotes §6.1's carried-gain mechanism from "the contribution" to a workaround for a mis-specified margin. The $2 \times 2$ sweep further shows DZ1, not DZ2, is dominant — correcting an emphasis this investigation itself had drawn from a single instance.

- **Experiment 5** (operator-acceptance study) is intentionally omitted, per spec §23's own framing of it as optional – noted here explicitly rather than silently dropped.

- **This benchmark instance rarely triggers SAR-CRP's repair path – but §6.1's lookahead margin, implemented and validated with real statistical power, now captures a real win when the trigger does recur.** SC4 (mean impact 0.090 vs. $\theta_{\text{impact}} = 0.30$) explains why Experiments 1/3/4's random event streams essentially never cross the trigger. Forcing the trigger directly (§6) showed the gap is not purely a benchmark-calibration artifact: at small scale, no repair's operational gain exceeds its own stability + confidence cost; at 50-container scale, a real gain is found on every seed but landed just under spec §9's 1%-of-$J_{old}$ fallback margin (Scenario B). An attempt to clear that margin by coordinating a fix across multiple simultaneous urgent containers (N6) did not succeed at 2+ containers, for an identified and documented reason. A multi-step experiment (Scenario C) then showed the margin itself, not the mechanism, was the binding constraint – and §6.1 implements the fix: a carried-gain lookahead margin that reproduces spec §9's original criterion exactly when there is nothing to remember. Scenario E validates it with real statistical power, not a single-seed anecdote: chaining a second, independently-built instance with its own sub-margin opportunity, plain `"sarcrp"` never updates (0/20 seeds) while `"sarcrp_lookahead"` updates on 18/20 ($p = 2.2 \times 10^{-5}$, Cliff's $\delta = -0.9$). This report does not retune $\theta_{\text{impact}}$, $\tau_{\text{frac}}$, $\lambda$, or the benchmark to manufacture a win at any point in this investigation – Scenario E's second instance was found by sweeping dimensions built with the identical deterministic construction recipe already used for instance A, keeping the one whose own gain independently reproduced instance A's sub-margin pattern. The remaining gap is now about how often real-world event streams present a *first* triggering opportunity at all (§20 SC4), not whether the lookahead mechanism delivers when a repeat opportunity exists – that question is now answered with a real, significant effect.

- `I_blocking` **(§8.4/§44.3, $w_b = 0.25$, tied for the largest impact weight) is structurally always 0 in this suite, which also means the impact score's own attainable ceiling is 0.75, not 1.0 – and honestly renormalizing changes real numbers, not just a rounding footnote.** Every call site passes the same physical state for both "old" and "new" because Paper 1's simulator never executes an action's physical effect between events (`commit_status` never becomes `"committed"` anywhere in `src/`) – giving this term real signal needs a physical-execution model, which is explicitly Paper 2's scope ("imperfect execution"), not Paper 1's. Ablation A6 (No Blocking Impact) is consequently a no-op vs. default SAR-CRP in this report, not evidence blocking pressure is unimportant. `sarcrp.impact_estimator.NORMALIZED_WEIGHTS` redistributes $w_b$'s mass across the four live terms ($w_o$=0.25, $w_t$=0.20, $w_p$=0.20, $w_c$=0.10 $\rightarrow$ 0.75) so the effective ceiling is honestly 1.0 again. Rerun on `REPORT_SEEDS`: mean_impact goes 0.087 → 0.116 (exactly /0.75, as expected when $I_{blocking}$=0 identically) and trigger_rate roughly triples, 5.3% → 12.1% – a real change, not noise, since some events sat in the $[0.225, 0.30)$ band that only clears $\theta_{\text{impact}} = 0.30$ after normalization. SC4 (mean impact in $[0.2, 0.8]$) still fails under both weightings, so the "trigger rarely fires on this random benchmark" finding survives the fix, but the true trigger rate on this benchmark is closer to 12% than the 4–5% a reader would compute from the nominal weights alone.

- **CRP_RL comparison caveat, now resolved with a direct test rather than left as a caveat.** §13's −48% figure rescores CRP_RL's own decisions under SAR-CRP's objective, not CRP_RL's native travel-time metric – raising the question of whether that gap is a genuine quality difference or an objective mismatch. Replaying both methods' move sequences through CRP_RL's own native cost engine (§13) answers this directly and with a nuance: CRP_RL loses even under its own metric on this project's flat-stack ($n_{rows}$=1) schema (a real quality gap there), but *wins* under its own metric on a genuine multi-row Lee instance matching its own training geometry – so the flat-stack schema is a genuine degenerate case for this model, not evidence of a general quality problem.

- **External validity (§15) is now partially addressed, not fully.** Two real Lee et al. instances (20 and 70 containers) confirm the same pattern found throughout this suite on data this project did not construct. Only two instances, both "random" type, were tried; the full Lee/Shin suite spans up to 720 containers and an "upside-down" variant, left to future work.

- **Base-paper citations** (Shin et al. 2026 [2], Zhou & Zhang 2024 [3], Zhang et al. 2025 [4]) were verified for DOI/venue in an earlier phase of this project. For this revision, Zhou & Zhang (2024) was re-read in full (not just DOI/venue) to assess it as a potential directly-competing baseline – see §14, which also adds three further citations (Sridharan et al. 1987, Goel & Wierman 2018, Cai et al. 2014) checked specifically for this revision, none previously in this report's citation list. Shin et al. 2026 and Zhang et al. 2025 were not re-verified beyond the earlier phase's DOI/venue check.

- **Zhang et al. 2025's citation is currently unearned:** it was assigned to `YardState.pickup_prob` (a retrieval-probability field, matching that paper's Retrieval Probability Matrix concept) as a base for future probabilistic-retrieval work, but a direct code check for this revision confirms `pickup_prob` is only ever copied between states (`freeze_horizon.py`, `local_search_repair.py`) – never read by `impact_estimator.py`, `objective.py`, or any decision logic in this report.

No experiment in this suite is actually affected by anything Zhang et al. 2025 proposes. The citation is disclosed here as scoped to future work (a natural fit for Paper 5's human-in-the-loop / operator-feedback setting, or an explicit probabilistic extension of the impact trigger), not removed, because it accurately signals intended future use – but it must not be read as evidence that this report's own results depend on or validate that paper's method in any way.

- **No directly competing named replanning-policy baseline from the CRP-specific literature was found or reimplemented.** §14 documents the search and the specific, checkable reason the closest candidate (SPFH, Zhou & Zhang 2024) does not qualify (different sub-problem: batch/tie-priority pickup-order optimization within one round, not a replanning-policy decision, with its own authors' stated factorial-complexity limit well below this report's 44–50-container scale). B1–B5 and CRP_RL together cover the policy-layer and solver-layer comparisons this literature family supports; a reviewer expecting a single named head-to-head competitor on this exact problem will not find one, because none was located.

# 18 Conclusion

The full baseline suite (B1–B6), all six ablations (A1–A6), the statistical protocol (spec §23.6), cross-layout generalization, ground truth/offline-proxy comparisons, and a real trained-model comparison (independently validated, §13) are now all implemented and exercised with real, seeded runs rather than placeholders. Six genuine implementation bugs were caught and fixed directly from this experiment suite's own output before being reported (the significance-test grid-cell aggregation bug, the MPC tail-state bug, an `I_blocking` spec-formula deviation, a local-search best-tracking regression, candidate C3/N5's own tail-state bug, and candidate scoring never actually checking plan validity) – all are documented above rather than silently corrected, and Experiment 1/3/4's numbers were re-verified unchanged after each fix.

The main finding, sharpened considerably by §6's existence-proof experiments, is no longer just "this benchmark's default event parameters keep SAR-CRP's trigger from firing" – forcing the trigger directly shows the mechanism's full pipeline (impact estimation, trigger, candidate generation, local search, fallback margin) works exactly as spec §9/§18 describe, and at industrially-relevant scale (50 containers) finds a real, positive improving candidate on every seed tested (Scenario B). An attempt to clear the remaining gap by coordinating multiple simultaneous urgent containers (N6) did not succeed, for a specific, identified reason rather than an unexplained one. A multi-step experiment (Scenario C) then showed the single-step fallback margin itself – not the trigger, not candidate generation, not local search – was the binding, myopic constraint: forcing through the exact candidate SAR-CRP already computes as best was strictly cheaper over a full episode on 19/20 seeds.

**That finding was then acted on, not just reported – and then validated with real statistical power, not a single seed.** §6.1 implements a carried-gain lookahead margin as a real mechanism (not Scenario C's one-time manual override), reproduces spec §9's original criterion exactly whenever there is nothing to remember, and was first validated by honestly sweeping all 20 `REPORT_SEEDS` (Scenario D), which found a real, substantial win (49.97) on one seed where a genuine second triggering opportunity naturally arose. Because one seed is not the standard of evidence a paper's central mechanism should rest on, Scenario E then built a second, independently-constructed instance (found by sweeping several dimension combinations with the same deterministic recipe already used for instance A, keeping the one that reproduced instance A's own sub-margin-gain pattern) to chain a genuine second opportunity on purpose, on every seed: plain `"sarcrp"` never updates there (0/20), `"sarcrp_lookahead"` updates on 18/20 ($p = 2.2 \times 10^{-5}$, Cliff's $\delta = -0.9$ – a large effect *within this specific, constructed two-instance chain*; it quantifies how reliably the mechanism wins once a real second opportunity has been arranged to exist, not an estimate of how large the benefit would be if sampled from a natural event-stream distribution, which §20's SC4 already shows presents such opportunities rarely in the first place). Separately, §15 confirms this suite's central pattern (SAR-CRP tying Static under default parameters, on this benchmark family) also holds on two real Lee et al. literature benchmark instances this project did not construct, ruling out "the benchmark was hand-picked" as an explanation.

Paper 1's central question is therefore no longer "does the mechanism work" (it does, end-to-end, as specified, and now demonstrably wins with real statistical power *when a repeat sub-margin opportunity is constructed to exist*) and no longer "can a harder benchmark be found" (tried at small and industrial scale, and on real literature data, without retuning any default parameter). **The contribution this report can defend is therefore scoped, not general: SAR-CRP's trigger+freeze+repair+lookahead pipeline is demonstrated to be correct and, conditional on a genuine recurring opportunity, to deliver a real, statistically significant improvement over the memoryless baseline – not a demonstrated general superiority over Static/FullReopt/Periodic/MPC on naturally-arising event streams, where the mechanism mostly ties with Static because the trigger rarely fires at all (§20 SC4).** What remains open is quantitative, not qualitative: how often a real deployment's own event stream presents a *first* triggering opportunity, let alone a second one to chain – §14 positions this pipeline against the closest available theory (rolling-horizon freezing, smoothed online optimization's switching-cost tradeoff, event-triggered control) in the absence of a directly competing named policy in the CRP-specific literature, rather than claiming a head-to-head win this report's own evidence does not support.

**The finding that supersedes all of the above.** §7 answers the question this report kept circling — why does a correctly implemented mechanism almost never find a worthwhile repair? — and the answer is not calibration, benchmark design, or search quality. It is that the objective as specified makes the mechanism's central tradeoff arithmetically impossible: a normalized delay term caps the achievable benefit at $\beta$, permanently below the $\alpha$ that one relocation costs (DZ1), and a relative margin outgrows that cap past $J_{old} = 50$ (DZ2). Zero positive gain across 190 events at $\theta_{\text{impact}} = \lambda = \mu = 0$ is not a weak result; it is the predicted one. Fixing the dimensional inconsistency changes repair feasibility from 0/16 instances to 14/16 with gains two orders of magnitude larger. **This reframes the paper: the contribution is a proven structural diagnosis of a whole class of stability-aware replanning objectives, plus two principled fixes, rather than a method that outperforms baselines.** It also generalizes past the CRP — rolling-horizon scheduling, MPC with switching costs, and event-triggered control applied to combinatorial problems all combine a bounded benefit term with a relative threshold, and inherit the same dead zones. Along the way this investigation twice drew a confident conclusion from a single instance and twice had to retract it after a systematic sweep; both retractions are recorded above rather than quietly corrected, because they are the clearest evidence for the methodological point the report argues.

**Stated plainly, as two separate contributions carrying two different kinds of evidence.** The trigger+freeze+repair+stability pipeline is an *infrastructure* contribution: specified, implemented, ablated, and validated end-to-end against a real independently-checked solver and real literature data – necessary groundwork this paper series builds on (Papers 2-5 assume it as a baseline), but empirically no better than Static on natural event streams, because the trigger itself rarely fires on this benchmark family, a finding that survives grounding event frequency in real terminal-operations data and does not weaken under it. The carried-gain lookahead margin is where this report's *validated* contribution actually lies, and it is validated twice, not once: §6.2

proves it is a bounded-overshoot, lazy accumulation-triggered threshold policy in the smoothed-online-optimization family, with explicit, checkable sufficiency conditions for when a cap or decay variant degenerates to permanent non-triggering; §6.1 then measures its real benefit with statistical power on a constructed opportunity, and the measured behavior matches what the proof predicts (the cap ablation broke the win exactly where Proposition 4 says it must; the decay ablation did not, exactly because Scenario E never reaches Proposition 5's asymptotic regime). A theoretical claim confirmed by an independent empirical measurement, rather than either standing alone, is the strongest form of evidence this report has to offer.

## References


[1] Y. Lee and Y. J. Lee, "A heuristic for retrieving containers from a yard," *Computers & Operations Research*, vol. 37, no. 6, pp. 1139–1147, 2010.

[2] W.-J. Shin, I. Choi, S.-H. Cho, and H.-J. Kim, "Learning to retrieve containers: A scale-diverse deep reinforcement learning approach for the container retrieval problem," *Transportation Research Part C: Emerging Technologies*, vol. 183, art. 105496, 2026. `https://doi.org/10.1016/j.trc.2026.105496`

[3] S. Zhou and Q. Zhang, "Real-Time Batch Optimization for the Stochastic Container Relocation Problem," *Applied Sciences*, vol. 14, no. 6, art. 2624, 2024. `https://doi.org/10.3390/app14062624`

[4] Z. Zhang, K. C. Tan, W. Qin, Y. Li, E. P. Chew, and K. Xu, "A Data-Driven Approach to Solving the Container Relocation Problem with Uncertainties," *Advanced Engineering Informatics*, vol. 65, art. 103112, 2025. `https://doi.org/10.1016/j.aei.2025.103112`

[5] S. V. Sridharan, W. L. Berry, and V. Udayabhanu, "Freezing the Master Production Schedule Under Rolling Planning Horizons," *Management Science*, vol. 33, no. 9, pp. 1137–1149, 1987. `https://doi.org/10.1287/mnsc.33.9.1137`

[6] G. Goel and A. Wierman, "Smoothed Online Optimization for Regression and Control," arXiv:1810.10132, 2018.

[7] R. Yaqoob, "A Recent Survey of Event Triggered Control of Nonlinear Systems," arXiv:2202.01684, 2022.

[8] B. Cai, S. Huang, D. Liu, and G. Dissanayake, "Rescheduling Policies for Large-Scale Task Allocation of Autonomous Straddle Carriers under Uncertainty at Automated Container Terminals," *Robotics and Autonomous Systems*, vol. 62, no. 4, pp. 506–514, 2014.